

Pemrograman Berorientasi Objek

Dany Candra Febrianto, S.Kom., M.Eng.

Minggu 03 : Class Diagram

Review Pertemuan Sebelumnya

- Visibility
 - default, private, public, protected
- Encapsulation
 - Setter, getter
- Constructor
 - Default Constructor, Parameter Constructor, Overloading Constructor
- Modularity
 - Package

Outline

- UML
- Modelling Class Diagram
- Relationship Class Diagram

Apa itu Unified Modeling Language (UML)

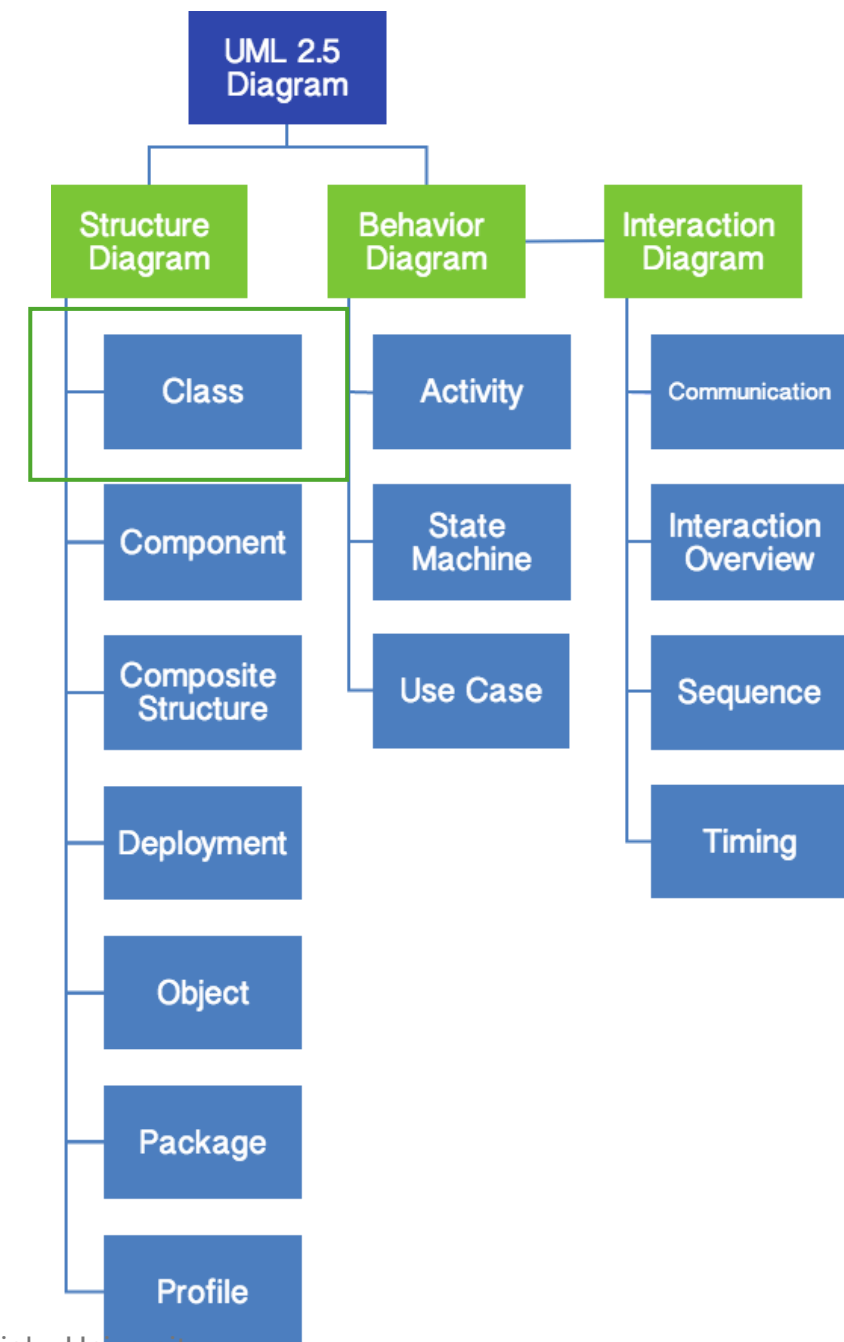
- Unified Modelling Language (UML) adalah sebuah "bahasa" yang telah menjadi standar dalam industri untuk visualisasi, merancang dan mendokumentasikan sistem informasi atau piranti lunak.
- UML menawarkan sebuah standar untuk merancang model sebuah sistem.
- Seperti bahasa-bahasa lainnya, UML mendefinisikan notasi dan syntax/semantik.
- Notasi UML merupakan sekumpulan bentuk khusus untuk menggambarkan berbagai diagram piranti lunak.
- Setiap bentuk memiliki makna tertentu, dan UML syntax mendefinisikan bagaimana bentuk bentuk tersebut dapat dikombinasikan.
- Notasi UML terutama diturunkan dari 3 notasi yang telah ada sebelumnya: Grady Booch OOD (Object-Oriented Design), Jim Rumbaugh OMT (Object Modeling Technique), dan Ivar Jacobson OOSE (Object-Oriented Software Engineering)

Diagram UML

Structure Diagrams : yaitu kumpulan diagram yang digunakan untuk menggambarkan suatu struktur statis dari sistem yang dimodelkan.

Behavior Diagrams : yaitu kumpulan diagram yang digunakan untuk menggambarkan perilaku sistem atau rangkaian perubahan yang terjadi pada sebuah sistem.

Interaction Diagrams : yaitu Kumpulan diagram yang digunakan untuk menggambarkan interaksi sistem dengan sistem lain maupun interaksi antar sub sistem pada suatu sistem.



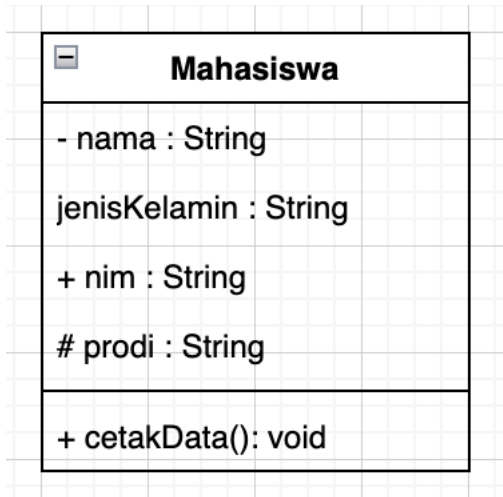
Mengapa UML..?

ASPEK	UML	DFD
Fokus	Pemodelan sistem berbasis objek (struktur dan perilaku)	Pemodelan aliran data antar proses
Pendekatan	Berorientasi objek	Berorientasi aliran data
Cocok untuk	Sistem dengan interaksi objek yang kompleks	Sistem informasi dan pemrosesan data
Diagram utama	Class Diagram, Use Case Diagram, Sequence Diagram	Diagram Aliran Data (Level 0, 1, 2)

Gunakan UML ketika bekerja pada sistem berbasis objek yang kompleks di mana Anda perlu memodelkan interaksi objek, perilaku, dan struktur sistem.

Gunakan DFD ketika fokus utama adalah aliran data dan pemrosesan informasi dalam sistem terstruktur, terutama untuk sistem berbasis pemrosesan data atau sistem lama.

Modeling Class



Class Diagram



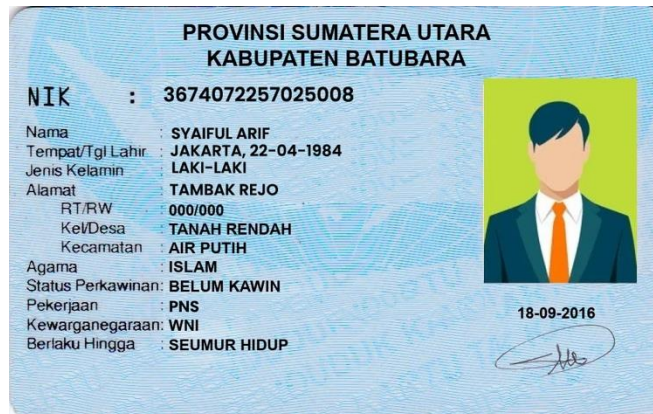
```
public class Mahasiswa {  
  
    private String nama;  
  
    String jenisKelamin;  
  
    public String nim;  
  
    protected String prodi;  
  
    public void cetakData() {  
        System.out.println(nama);  
        System.out.println(nim);  
        System.out.println(prodi);  
        System.out.println(jenisKelamin);  
    }  
}
```

Implementasi
Code Java

Latihan membuat class

Buatlah class diagram dan Java Class Berdasarkan kasus berikut:

Kita akan membuat sistem untuk pendataan penduduk, dimana data penduduk bisa kita lihat, tambah, ubah dan hapus. Berikut adalah contoh data penduduk yang akan digunakan dalam sistem.



Relationship Class Diagram

Jenis Relasi berdasarkan sifatnya:

- Has A :
Sebuah kelas memiliki objek dari kelas lain sebagai bagian dari dirinya.
- Is A :
Sebuah kelas adalah turunan dari kelas lain (hubungan generalisasi).
- Uses :
Sebuah kelas menggunakan kelas lain untuk menjalankan fungsinya.

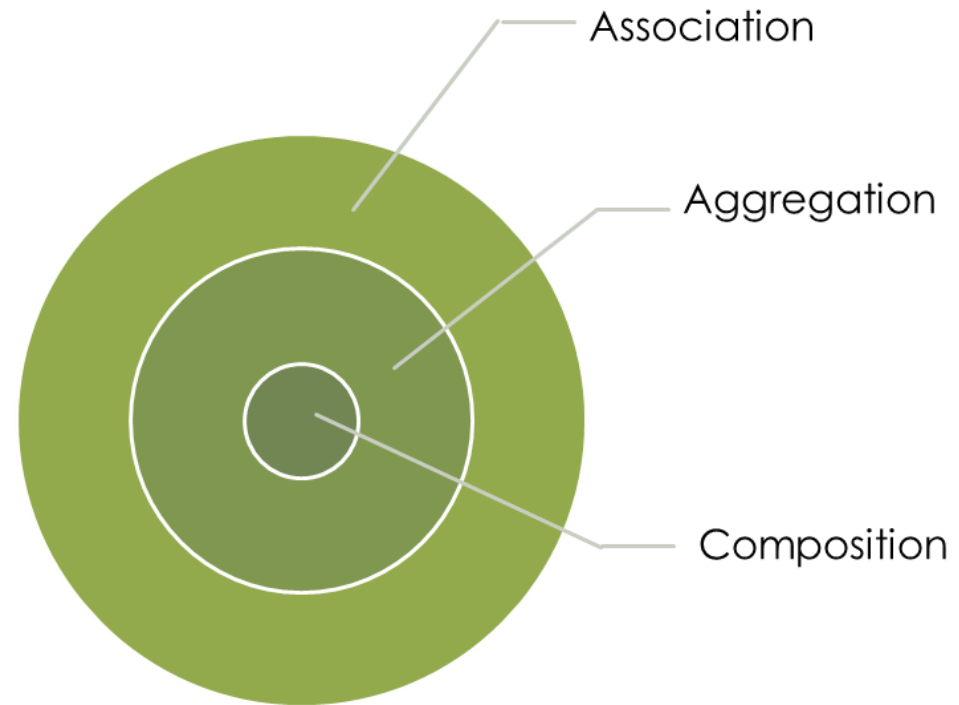
Multiplicity / Cardinality

Pada relasi terdapat suatu penanda yang disebut multiplicity. Multiplicity ini akan mengindikasikan berapa banyak obyek dari suatu kelas terelasi ke obyek lain. Notasi UML untuk multiplicity ini adalah sebagai berikut:

<i>Multiplicity</i>	Arti
*	Banyak
0	Nol
1	Satu, bisa ditulis bisa tidak
0..*	Antara nol sampai banyak
1..*	Antara satu sampai banyak
0..1	Nol atau 1
1..1	Tepat satu

Basic Relationship

- Association
- Aggregation
- Composition



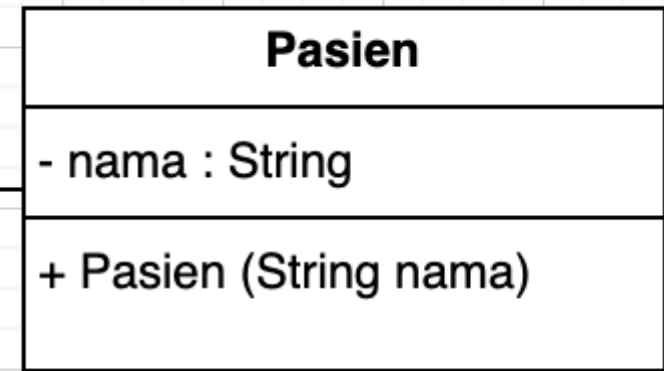
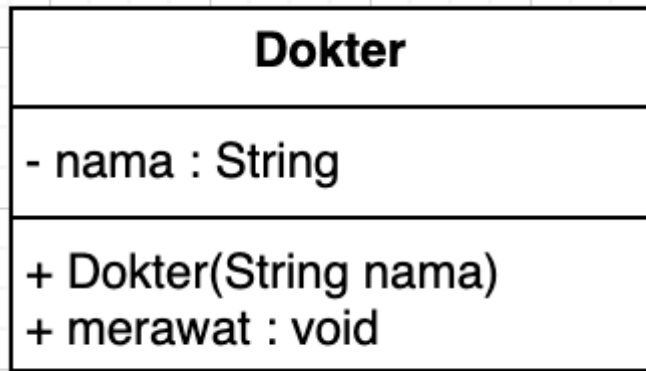
Association



Association

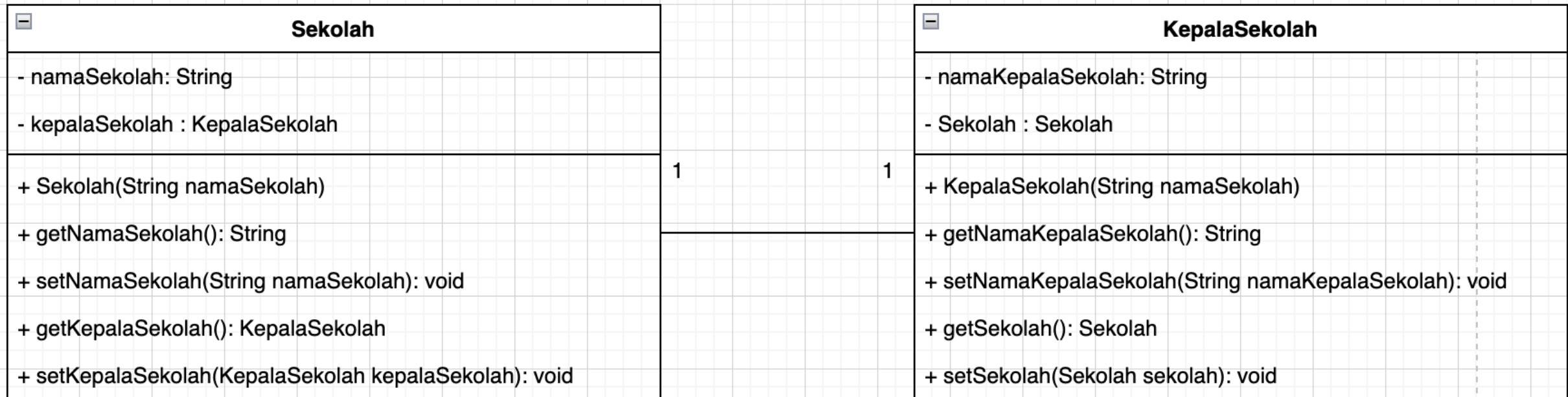
- Relasi *Association* menyatakan relasi antar dua kelas terpisah yang relasinya terjadi melalui pertemuan objek dari masing-masing kelas di bagian atributnya.
- Jika dua kelas dalam sebuah model perlu berkomunikasi satu sama lain, maka harus ada hubungan (link) di antara mereka.
- Asosiasi antara dua kelas menunjukkan bahwa objek di salah satu ujung asosiasi "mengenal" objek di ujung lainnya dan dapat mengirim pesan kepada mereka.

Contoh Assocoation



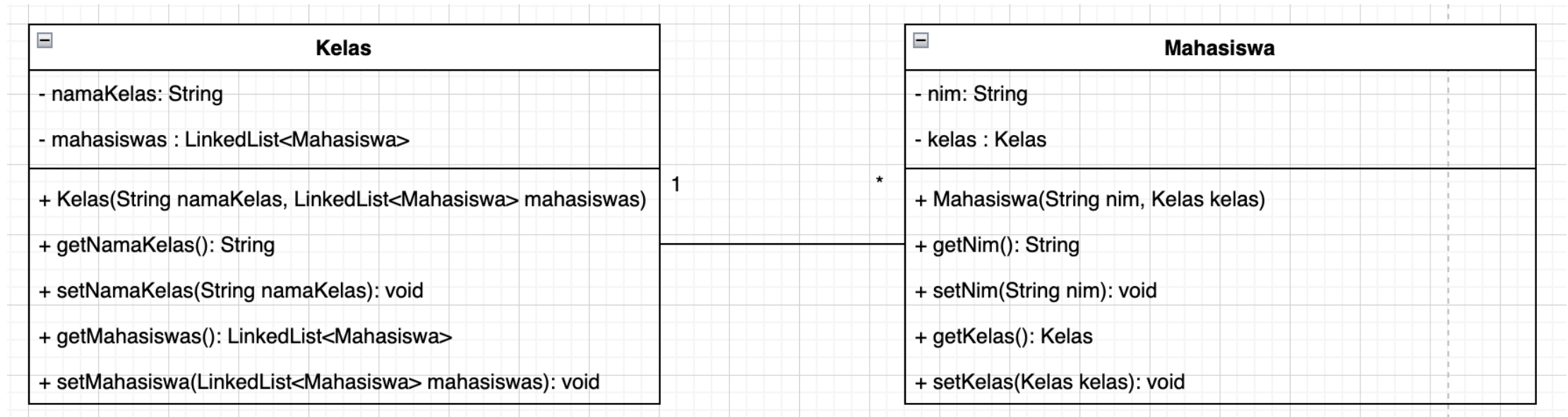
Dokter terasiasi untuk merawat pasien

Contoh Association Tepat Satu (One To One)



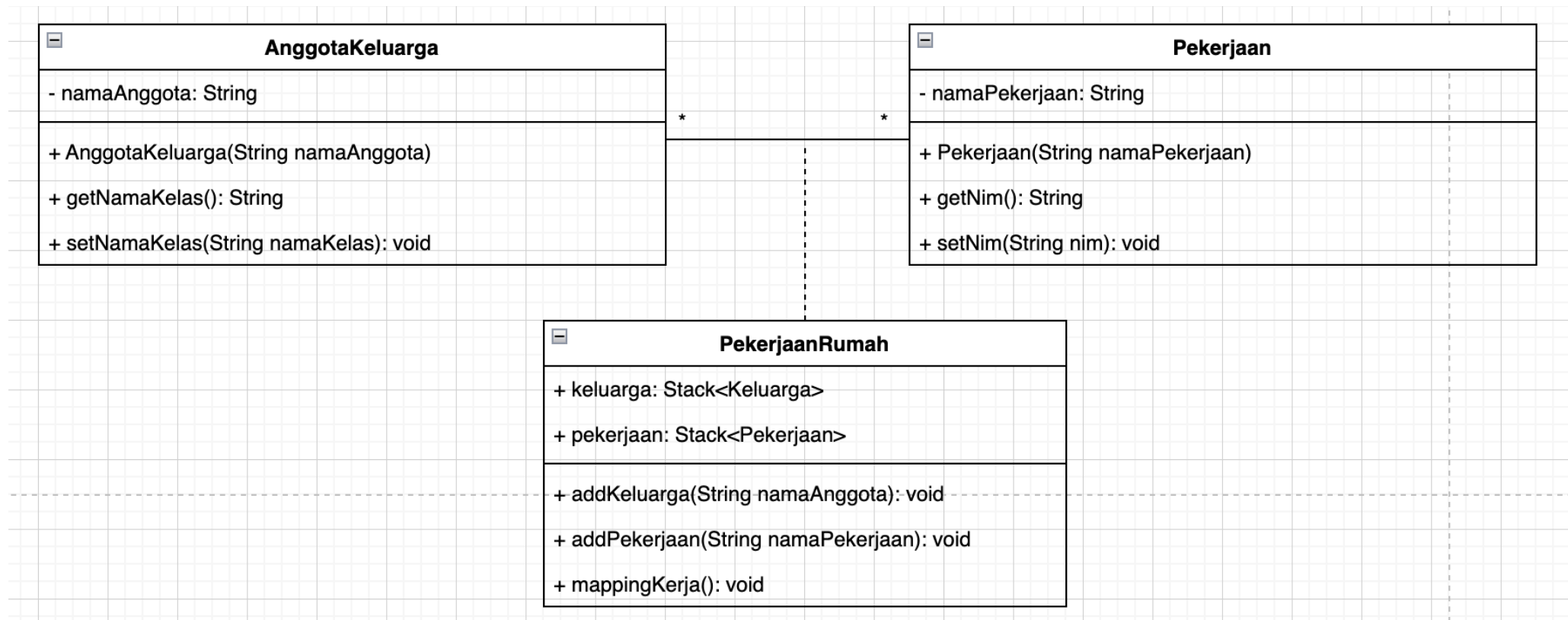
One To One : Karena setiap 1 sekolah memiliki 1 kepala sekolah

Contoh Association Satu ke Banyak (One To Many)



One To Many : Karena setiap 1 kelas memiliki banyak mahasiswa

Contoh Association Banyak ke Banyak (Many To Many)



One To Many : Karena setiap 1 kelas memiliki banyak mahasiswa

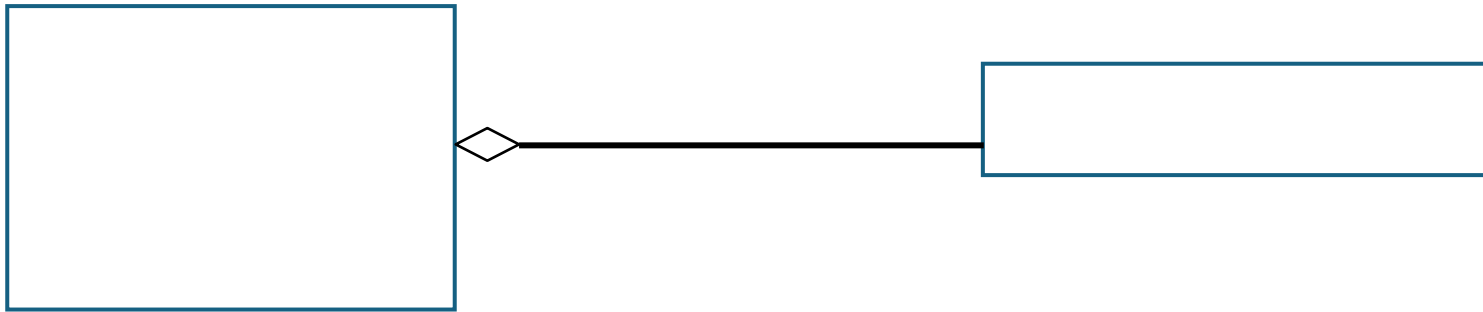
Aggregation



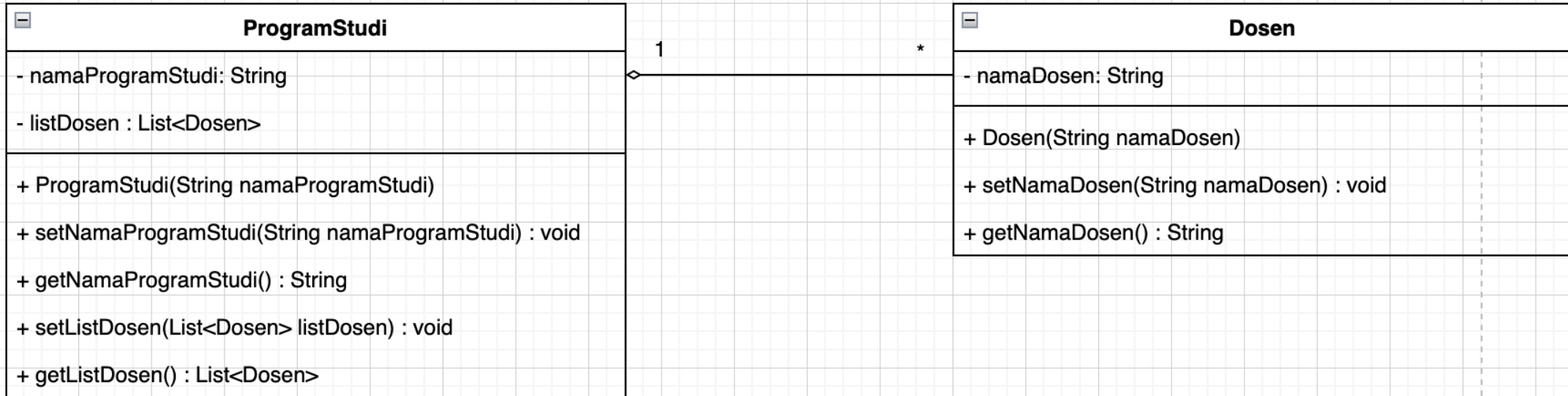
Aggregasi (Aggregation)

Relasi aggregasi adalah suatu bentuk relasi yang jauh lebih kuat dari pada asosiasi. Aggregasi dapat diartikan bahwa suatu kelas merupakan bagian dari kelas yang lain namun bersifat tidak wajib. (Sifat Has-A)

Simbol



Contoh Agregasi

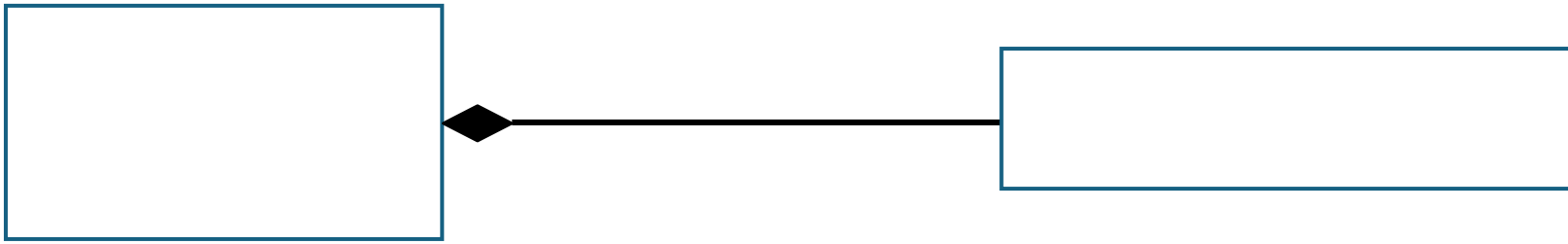




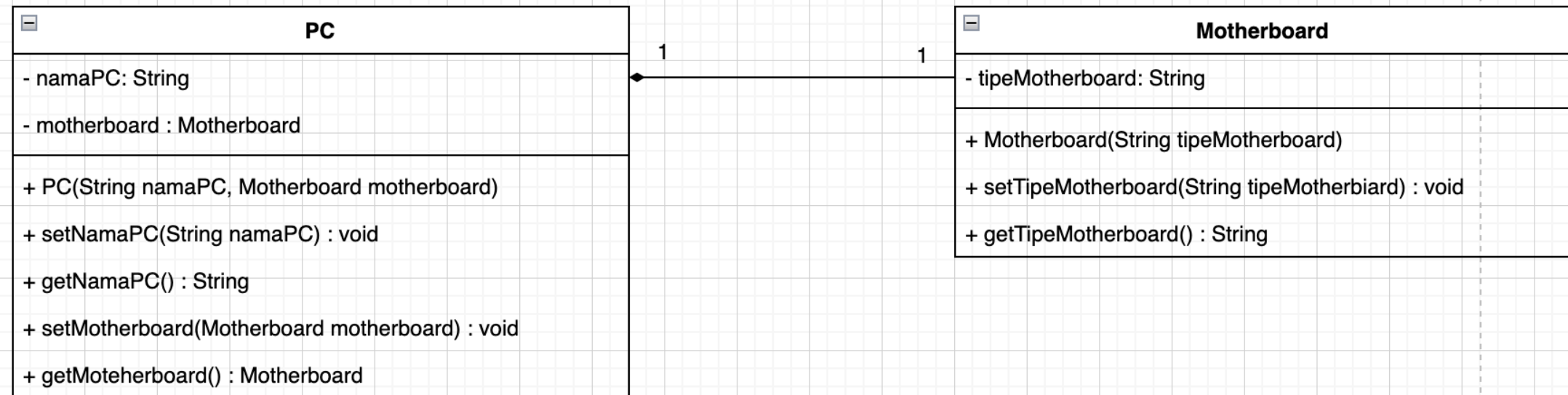
Komposisi (Composition)

Relasi ini merupakan relasi yang paling kuat dibandingkan dengan asosiasi dan agregasi. Pada komposisi diartikan bahwa suatu kelas merupakan bagian yang wajib dari kelas yang lain.
(Sifat Part-of / owns -a)

Simbol



Contoh Komposisi



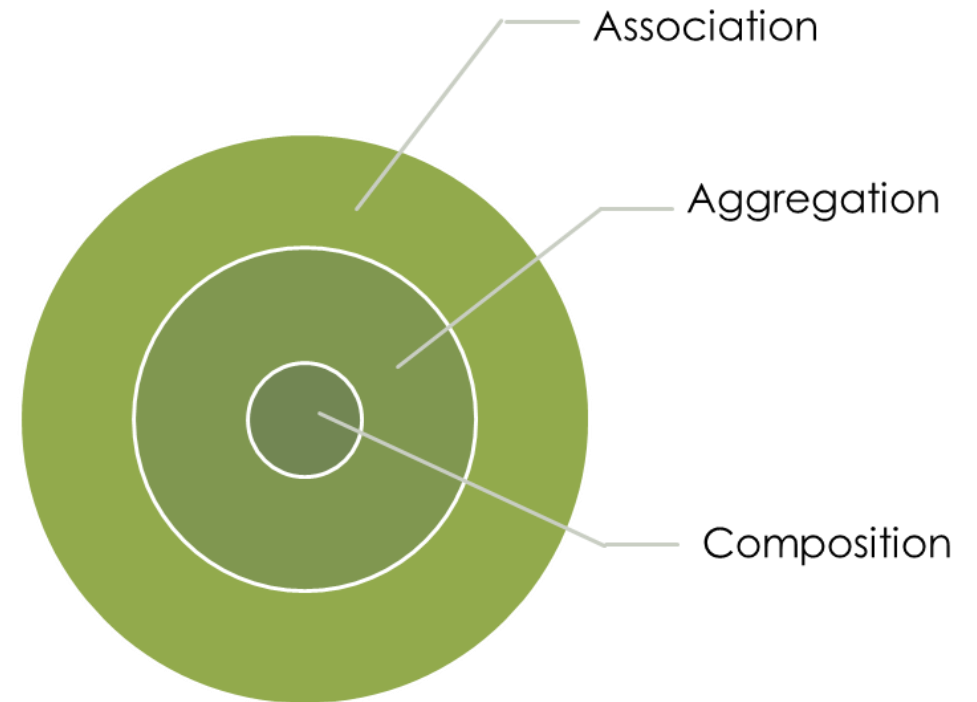
Komposisi : PC wajib memiliki motherboard dapat dilihat dari konstruktor PC() Dimana kelas motherboard di inisias

Hubungan Association, Aggregation, Composition

Relasi Association menjelaskan konsep relasi yang terjadi antar kelas dengan pertukaran objek,

Relasi Aggregation menjelaskan tentang konsep kepemilikan (Has-A) tanpa memperngaruhi siklus hidup dari objek yang direalisasikan,

Relasi Composition menjelaskan konsep objek yang berelasi merupakan bagian dari kelas lainnya.



Ilustrasi Kasus Relasi (Agregasi, Komposisi)

